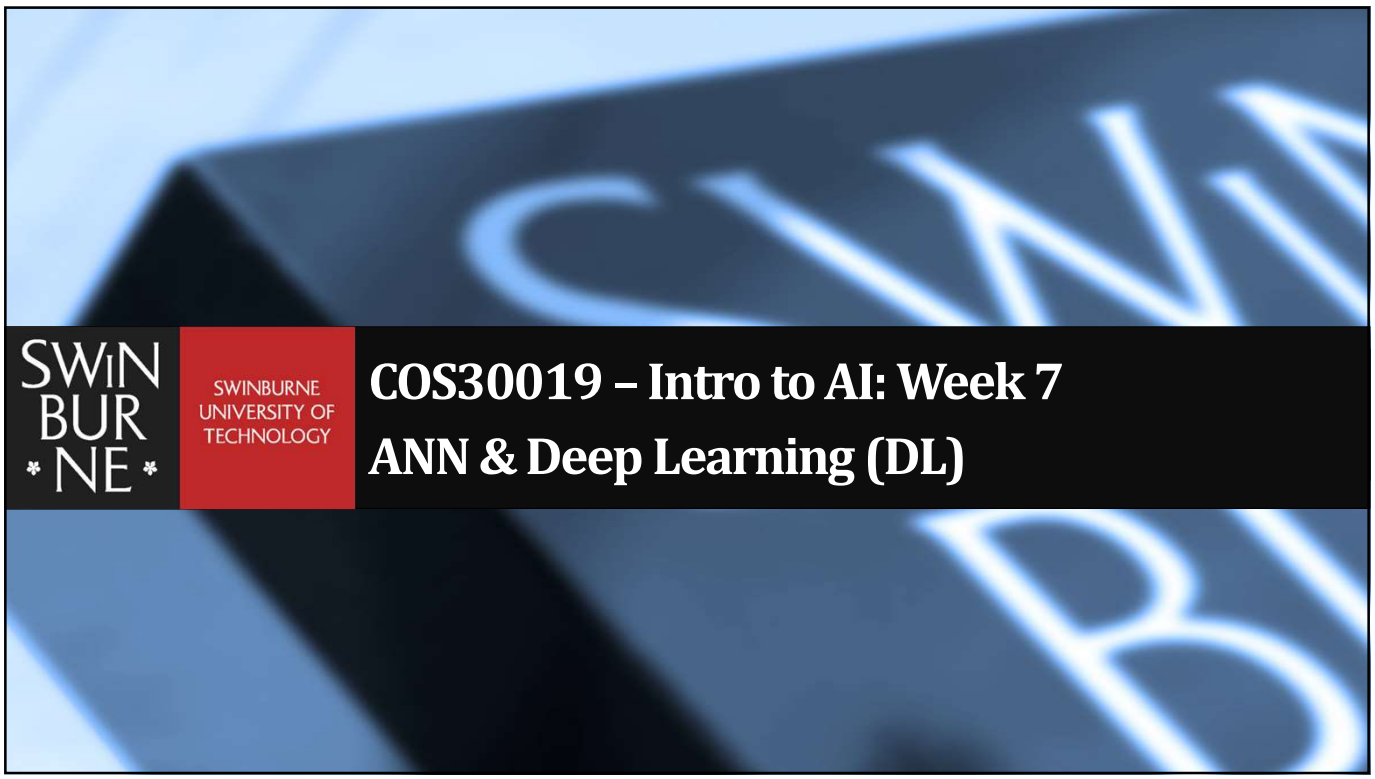


1

Outline

- **What is** and **why** Deep Learning (**DL**)?
  - Why now?
- Logistic Regression
  - Logistic regression for different image processing tasks
- Things are not always simple
- Artificial Neural Networks
  - Deep Learning (DL)
- Learning in Multiple-Layer Neural Networks
  - Back Propagation Algorithm

2

Why DL?

Traditional ML:

ML – How does Machine learn?

Features/ Attributes

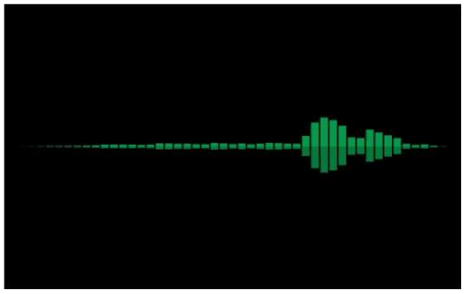
Target

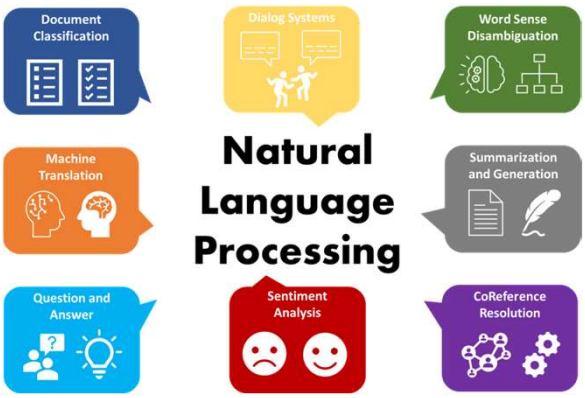
The "play tennis" dataset

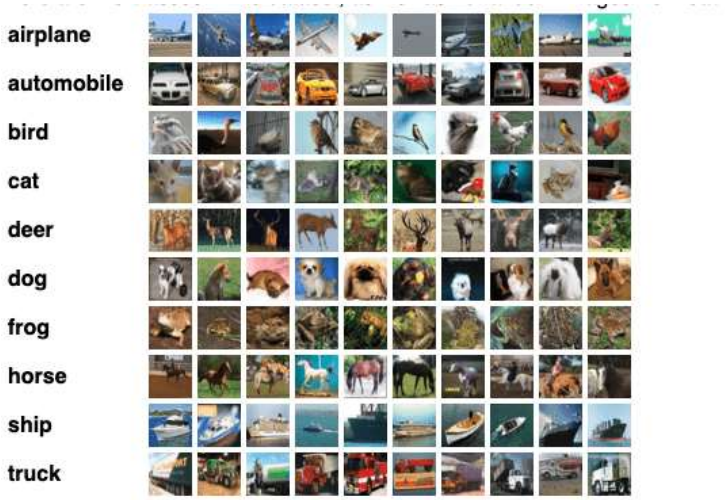
Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Source: Tom Mitchell, 1997

3



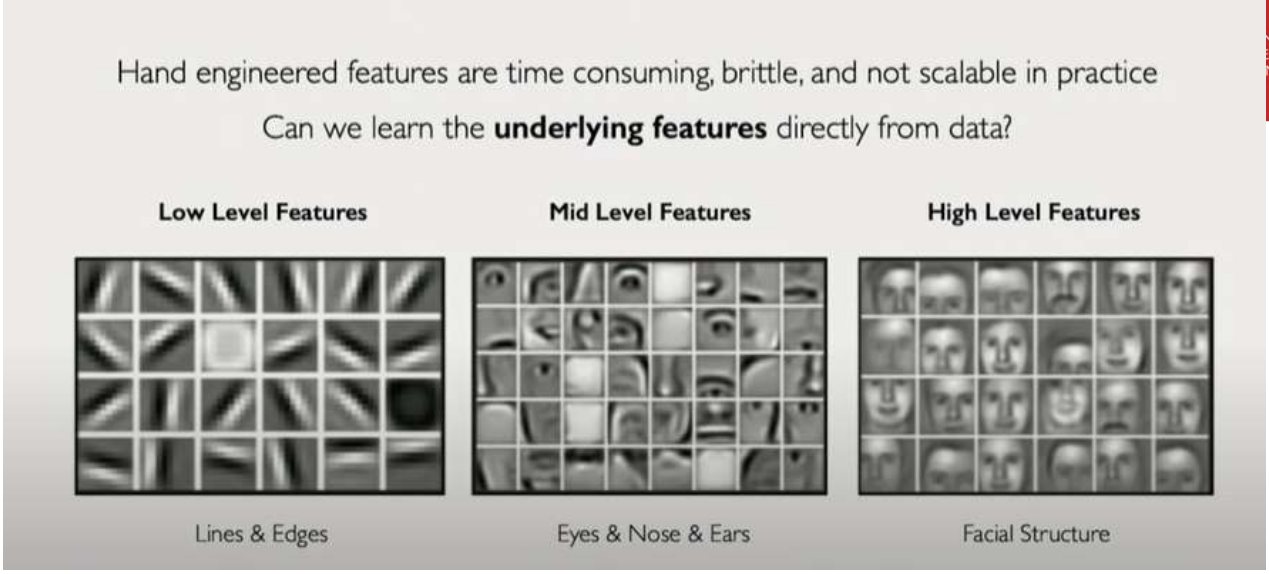




Which features? How to determine them?

4

Why DL?

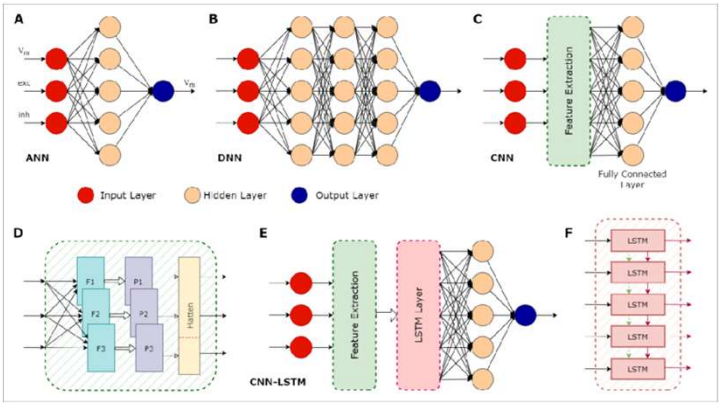


Source: **MIT Introduction to Deep Learning | 6.S191**

5

Why now?

- Better algorithms & understanding
- Computing power (GPUs, TPUs, ...)
- Data with labels
- Open-source tools and models

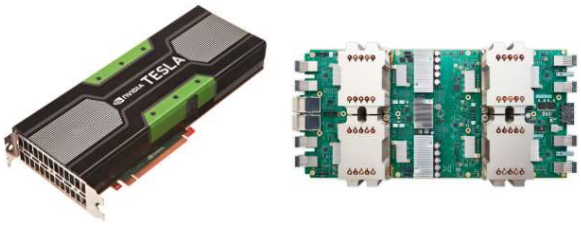


Source: **https://www.ip-paris.fr/education/masters/mention-mathematiques-appliquees-statistique/master-year-2-data-science**

6

Why now?

- Better algorithms & understanding
- Computing power (GPUs, TPUs, ...)
- Data with labels
- Open-source tools and models



GPU and TPU

Source: **https://www.ip-paris.fr/education/masters/mention-mathematiques-appliquees-statistique/master-year-2-data-science**

7

Why now?

- Better algorithms & understanding
- Computing power (GPUs, TPUs, ...)
- Big data
- Open-source tools and models

Source: **https://www.ip-paris.fr/education/masters/mention-mathematiques-appliquees-statistique/master-year-2-data-science**

8

Why now?

- Better algorithms & understanding
- Computing power (GPUs, TPUs, ...)
- Data with labels
- Open-source tools and models

Source: **https://www.ip-paris.fr/education/masters/mention-mathematiques-appliquees-statistique/master-year-2-data-science**

9

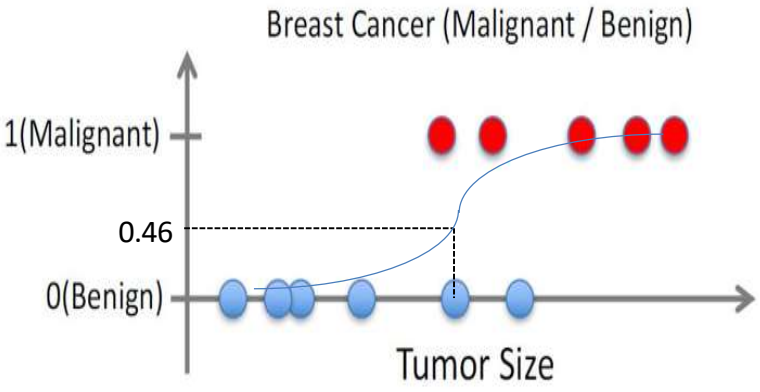
What is DL?

Deep Learning (DL)

- **Deep Neural Networks (DNNs)**
  - **(Artificial) Neural Networks (ANNs)**
    - **Neurons**
      - **Perceptron**
      - **Logistic Regression**

10

Logistic regression



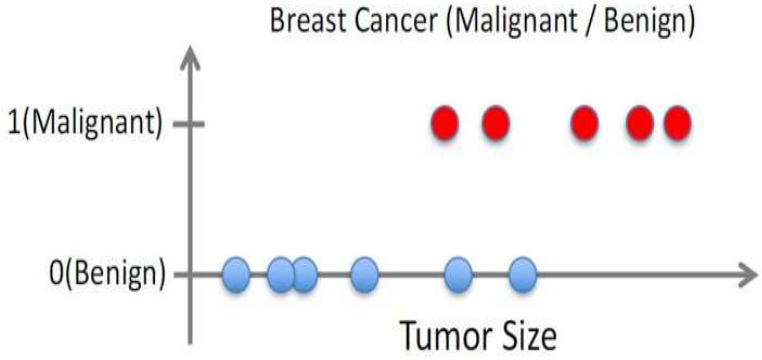
Source: Andrew Ng/Stanford Online

One way to get the desired classification is by fitting a symmetric curve (dashed blue one). Then a **tumor size** can map to that curve and give us a value between 0 and 1 → the **probability that the tumor of that size is malignant**!

11

Logistic regression

- It is actually a classification technique!
- Remember this classification problem?

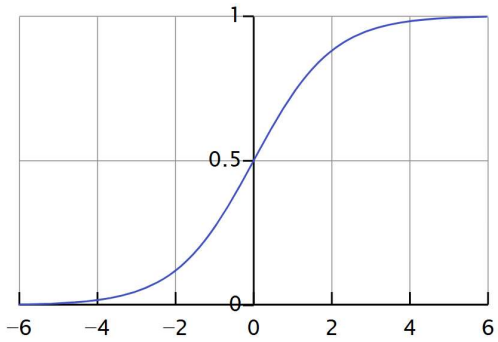


Source: Andrew Ng/Stanford Online

12

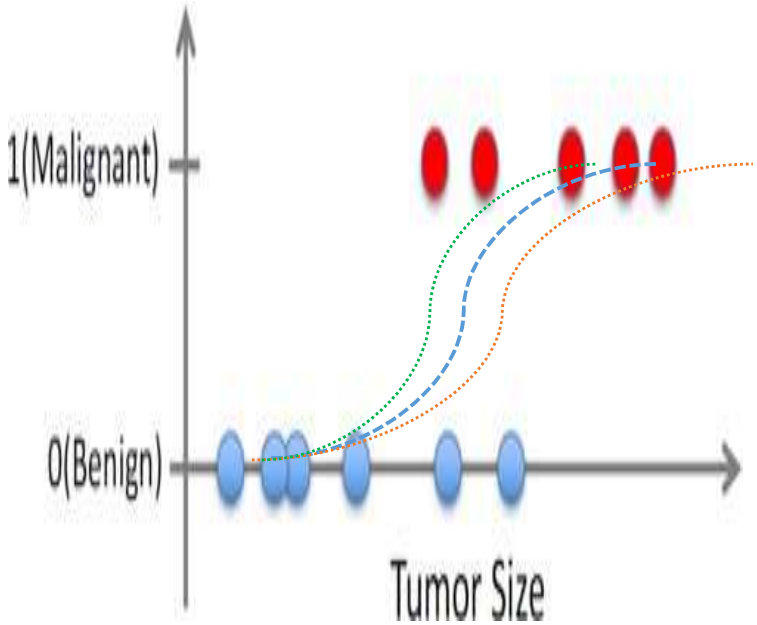
- A function with this shape is called **logistic function** or **sigmoid function:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



- As we don't want to return the probability (requiring some interpretation), what we can do is to map this probability outcome to the classification:
  - when the **prob ≥ 0.5**, the tumor will be *classified by our model* as **Malignant**; **else** it will be classified as **Benign**.

13



- Note that the blue model is not the only one. There are many. We still face the problem of which logistic regression model is the "right" one → **minimize the cost function**

14

Logistic regression - mathematically

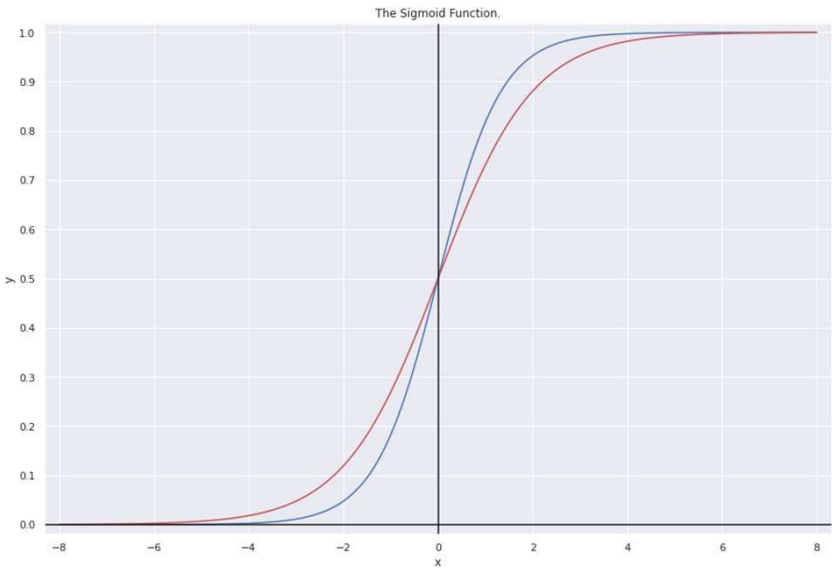
- $y \sim f(x) = \frac{1}{1 + e^{-g(x)}} = \sigma(g(x))$
- Well, more precisely $\text{Prob}(y=1 | x) = \frac{1}{1 + e^{-g(x)}}$
- E.g., $\text{Prob}(y = \text{Malignant} | \textit{size}) = \frac{1}{1 + e^{-g(\textit{size})}}$
- and $g(x) = \mathbf{w_0} + \mathbf{w_1} * x$
- Of course, for multivariate problem:

$$g_W(x_1,\ldots, x_m) = \mathbf{w_0} + \mathbf{w_1} * x_1 + \ldots + \mathbf{w_m} * x_m = \mathbf{Wx} \quad (x_0=1)$$

- Some other notations: $g_{W,b}(x_1,\ldots, x_m) = \mathbf{b} + \mathbf{w_1} * x_1 + \ldots + \mathbf{w_m} * x_m$

15

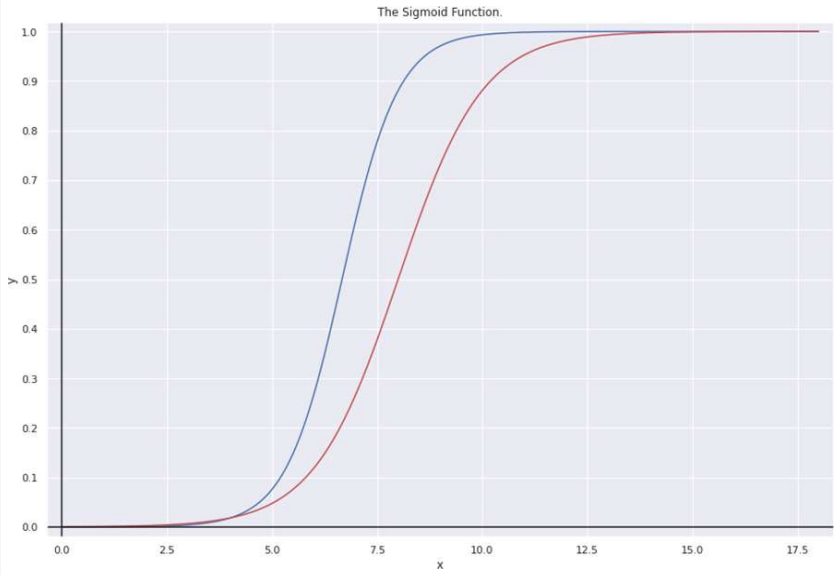
Logistic regression - examples



- The blue model is the function sigmoid(1.5*x), while the red line is the function sigmoid(x)

16

Logistic regression - examples



- The blue model is the function sigmoid(-10+1.5*x), while the red line is the function sigmoid(-8 + x)

17

Logistic regression – more maths

- So, what would a cost function look like for Log.Reg?

$$J(W) = -\frac{1}{n}\sum_{i=1}^{n} \left(y^i log\sigma(g(x^i)) + (1 - y^i)\log(1 - \sigma(g(x^i))) \right)$$

  - Note the minus (-) sign at the beginning
  - The expression after the sum is called the (log) likelihood of the parameters W (i.e., how likely does the model (parameterized by W) correctly classify the target variable y based on the input vars x)
  - Method: **Maximum Likelihood Estimation (MLE)**. The mathematical intuition of the above can be found in Andrew Ng's video:

https://www.youtube.com/watch?v=SHEPb1JHw5o

18

Logistic Regression for Image classification

Task 1: Determine if a dog is in the image: $\begin{cases} 1 & \text{if there is a dog} \\ 0 & \text{if there isn't a dog} \end{cases}$

(32x32x3)

$(x_0 = 1)$, x_1, x_2, $\vdots$, x_{m-1}, x_m → Wx | σ → $\hat{y} = \sigma(Wx)$

General steps in a Logistic Regression algorithm:

- Initialise (randomly) W
- Find the optimal W^* ($W = W - \alpha \frac{\partial J(W)}{\partial W}$ over a number of iterations)
- Use W^* for inference: Given an input x, $\hat{y} = \sigma(W^* x)$

19

Logistic Regression for Image classification

Task 1: Determine if a dog is in the image: $\begin{cases} 1 & \text{if there is a dog} \\ 0 & \text{if there isn't a dog} \end{cases}$

(32x32x3)

$(x_0 = 1)$, x_1, x_2, $\vdots$ → (artificial) neuron → $\hat{y} = \sigma(Wx)$

More generally:

Neuron = Linear + Activation

(& there are many **activation functions**: Sigmoid, step, linear, tanH, Gaussian, ReLU, etc.)

General steps i

- Initialise (ra
- Find the optimal W^* ($W = W - \alpha \frac{\partial J(W)}{\partial W}$ over a number of iterations)
- Use W^* for inference: Given an input x, $\hat{y} = \sigma(W^* x)$

20

Logistic Regression for Image classification

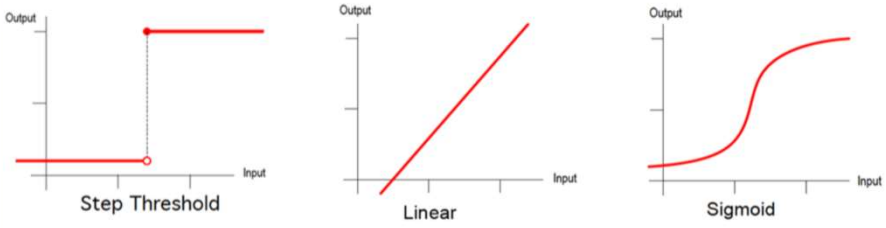
Task 1: Determine if a dog is in the image: $\begin{cases} 1 & \text{if there is a dog} \\ 0 & \text{if there isn't a dog} \end{cases}$

(32x32x3)

$(x_0 = 1)$
x_1
x_2

(artificial) neuron

$y = \begin{cases} A_1, if\ w.x + b > 0 \\ A_0, \quad otherwise \end{cases}$ $y = \sum_{i=1}^{N} x_i w_i$ $y = \frac{1}{1+e^{-x}}$



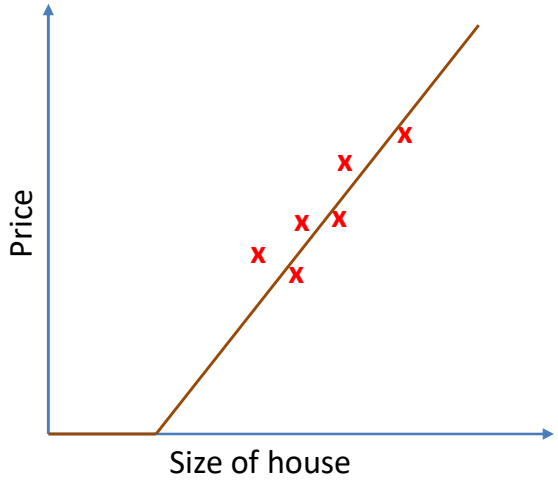
General s

- Initiali
- Find th
- Use *W*

Learn more about activation functions - https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0

21

Price Prediction –Single Neuron



Rectified **L**inear **U**nit

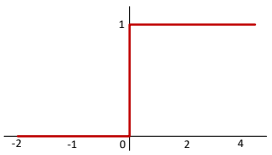
22

Perceptron

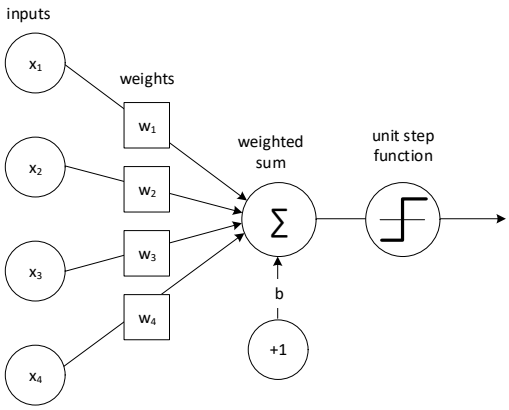
- Simplest neural network possible
- Single neuron that performs binary classification
- Uses a *step activation function*
  - Also referred to as *Heaviside Step Function*

$$f(x) = \begin{cases} 1, & if\ w.x + b > t \\ 0, & otherwise \end{cases}$$

where $w.x$ is the dot product $\sum_{i=1}^{N} x_i w_i$ and t is the threshold



Step Activation Function



23

What about Multinomial Classification?

(aka. Multiclass classification)

Task 2: Classifying images into cat/dog/bird?

(32x32x3)

$$(x_0 = 1),\ x_1,\ x_2,\ \vdots,\ x_{m-1},\ x_m$$

$W^{[1]}x|\sigma \rightarrow \hat{y}_c = \sigma(W^{[1]}x)$

$W^{[2]}x|\sigma \rightarrow \hat{y}_d = \sigma(W^{[2]}x)$

$W^{[3]}x|\sigma \rightarrow \hat{y}_b = \sigma(W^{[3]}x)$

Label: $\begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$

(dog)

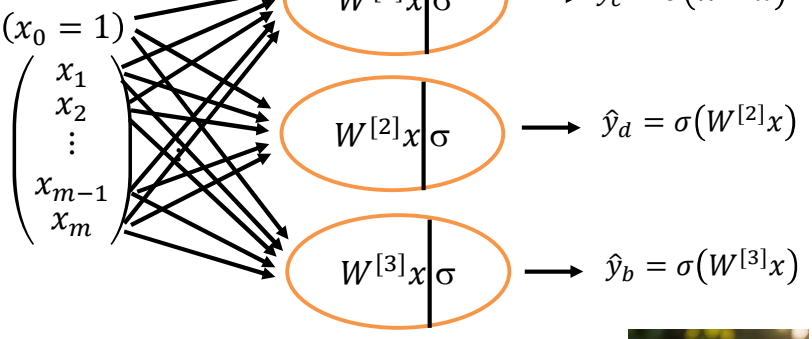
24

What about Multinomial Classification?

(aka. Multiclass classification)

Task 2: Classifying images into cat/dog/bird?

(32x32x3)



Label: $\begin{pmatrix}0\\1\\0\end{pmatrix}$

(dog)

And what about this example + label?

Task 2': Is there a cat/dog/bird in an image.

$\begin{pmatrix}1\\1\\0\end{pmatrix}$

(dog & cat)

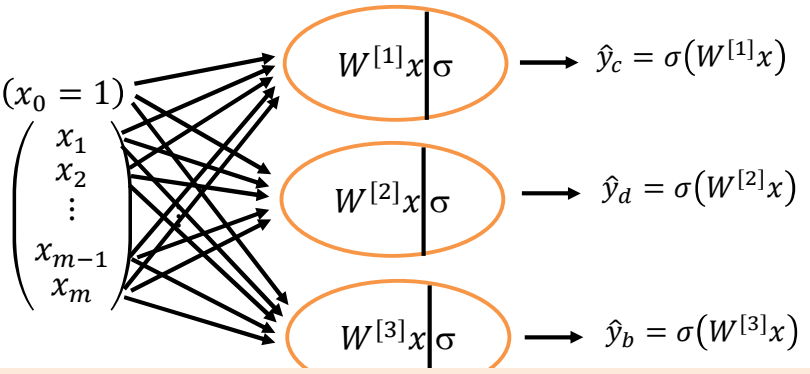
25

What about Multinomial Classification?

(aka. Multiclass classification)

Task 2: Classifying images into cat/dog/bird?

(32x32x3)



Cost function:

$$J(W) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{3}\left(y^i \log\left(\sigma\left(W^{[j]}x^i\right)\right) + (1-y^i)\log\left(1-\sigma\left(W^{[j]}x^i\right)\right)\right)$$

26

Multinomial Classification under constraints

Task 3: Which animal (a cat, dog or bird) is in an image (and at most one of them)?

(32x32x3)

The **softmax multiclass regression**

$$(x_0 = 1)\quad \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{m-1} \\ x_m \end{pmatrix}$$

$$Z^1 = W^{[1]}x \qquad Z^2 = W^{[2]}x \qquad Z^3 = W^{[3]}x$$

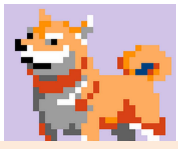
$$\hat{y} = \begin{pmatrix} \frac{e^{Z^1}}{\sum_{i=1}^{3} e^{Z^i}} \\ \frac{e^{Z^2}}{\sum_{i=1}^{3} e^{Z^i}} \\ \frac{e^{Z^3}}{\sum_{i=1}^{3} e^{Z^i}} \end{pmatrix}$$

The **softmax** function

27

Multinomial Classification under constraints

Task 3: Which animal (a cat, dog or bird) is in an image (and at most one of them)?



The **softmax multiclass regression**

$$(x_0 = 1)\quad \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x \end{pmatrix}$$

$$Z^1 = W^{[1]}x \qquad Z^2 = W^{[2]}x \qquad W^{[3]}x$$

$$\hat{y} = \begin{pmatrix} \frac{e^{Z^1}}{\sum_{i=1}^{3} e^{Z^i}} \\ \frac{e^{Z^2}}{\sum_{i=1}^{3} e^{Z^i}} \\ \frac{e^{Z^3}}{\sum_{i=1}^{3} e^{Z^i}} \end{pmatrix}$$

The **softmax** function

Cost function:

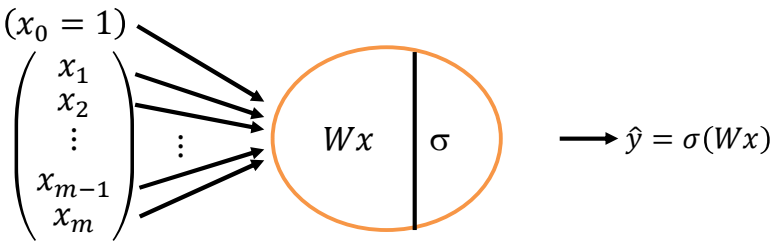
$$J(W) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{3} y_{[j]}^{i}\log(\hat{y}_{[j]})$$

28

Logistic Regression for Image classification

Task 1: Determine if a dog is in the image: $\begin{cases} 1 & \text{if there is a dog} \\ 0 & \text{if there isn't a dog} \end{cases}$

(32x32x3)



General steps in a Logistic Regression algorithm:

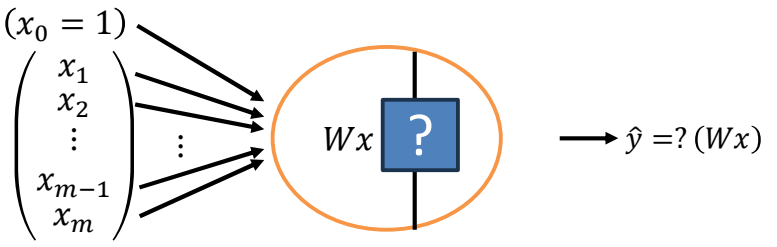
- Initialise (randomly) W
- Find the optimal W^* ($W = W - \alpha \frac{\partial J(W)}{\partial W}$ over a number of iterations)
- Use W^* for inference: Given an input x, $\hat{y} = \sigma(W^* x)$

29

Machine Learning for Image processing

Task 1': Determine the age of the dog is in the image?

(32x32x3)



30

Machine Learning for Image processing

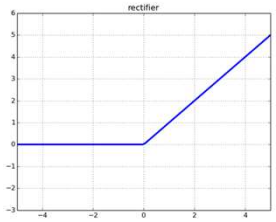
Task 1': Determine the age of the dog is in the image?

(32x32x3)

$$(x_0 = 1),\ x_1,\ x_2,\ \vdots,\ x_{m-1},\ x_m$$

Wx ?

$\hat{y} = ?\ (Wx)$



Remember the ReLU activation function?

Rectified **L**inear **U**nit

31

Here come the big question…

- Would it suffice to have just linear + activation (with the right activation function, of course)
  - Classification? ✓
  - Regression? ✓
- Then should we complicate our lives with neurons and neural networks?

32

Things are not always simple

- There can be hundreds/thousands/ even millions of people we need to recognise
- There are millions of species of animals

33

Things are not always simple

- There are thousands of languages used by people around the world
  - Different syntaxes and grammars
  - Different rules

34

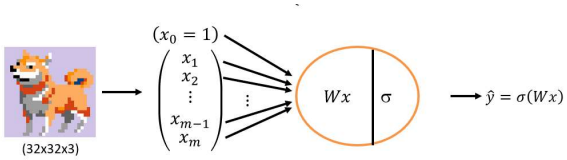
Things are not always simple

- Even in a single language (e.g., English), voice recognition systems have to deal with many different accents

35

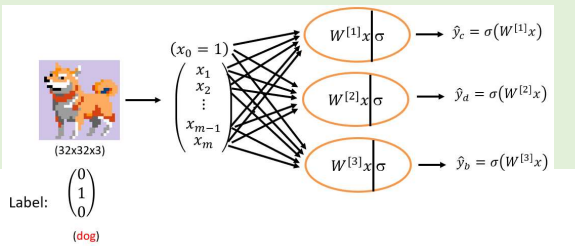
Here come the big question…

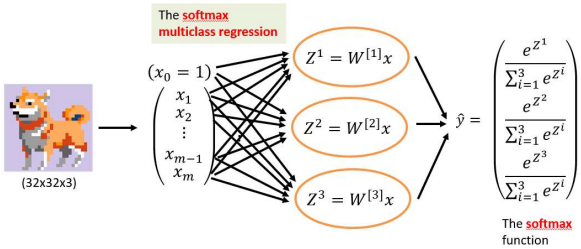
- Would it suffice to activation function the right



We have seen different kity:

- Task 1 (and 1')
- Task 2 (and 2')
- Task 3





36

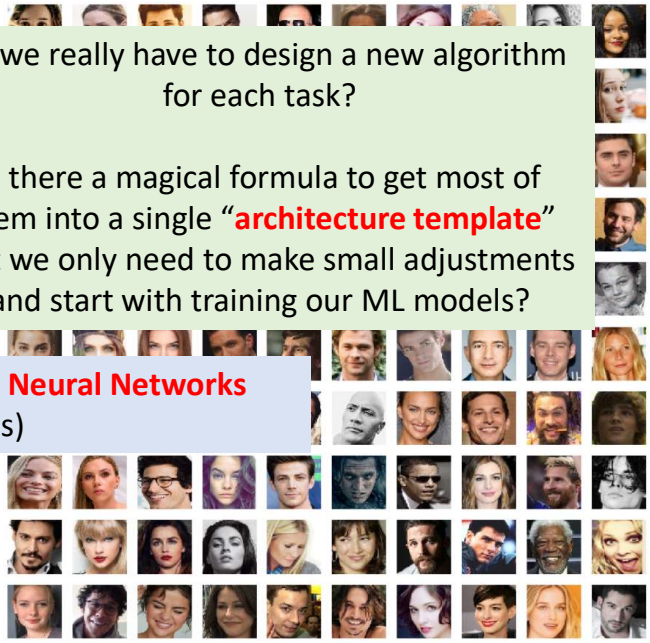
These can be your task 100th, 157th, …

- There can be hundreds/thousands/ even millions of people we need to recognise
- There are millions of specie

Do we really have to design a new algorithm for each task?

Is there a magical formula to get most of them into a single "**architecture template**" that we only need to make small adjustments and start with training our ML models?

Welcome to **Artificial Neural Networks** (ANNs)



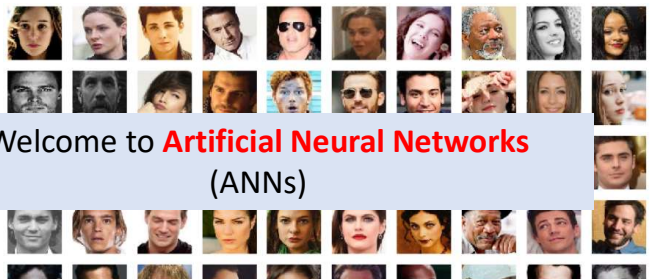
SWINBURNE UNIVERSITY OF TECHNOLOGY

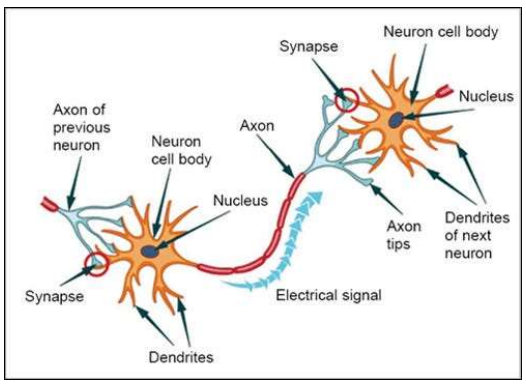
37

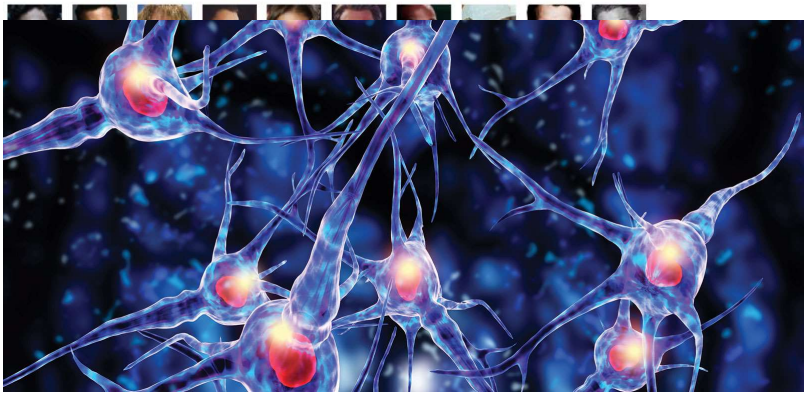
These can be your task 100th, 157th, …

- There can be hundreds/thousands/ even millions of people we need to recognise

Welcome to **Artificial Neural Networks** (ANNs)



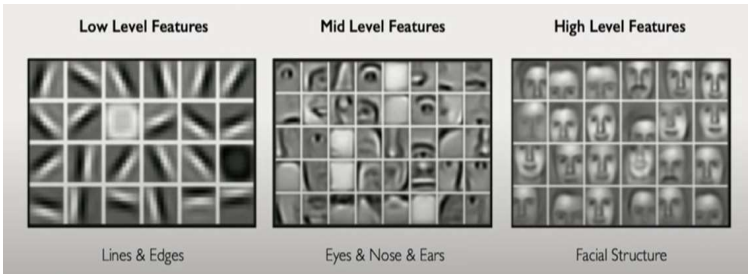


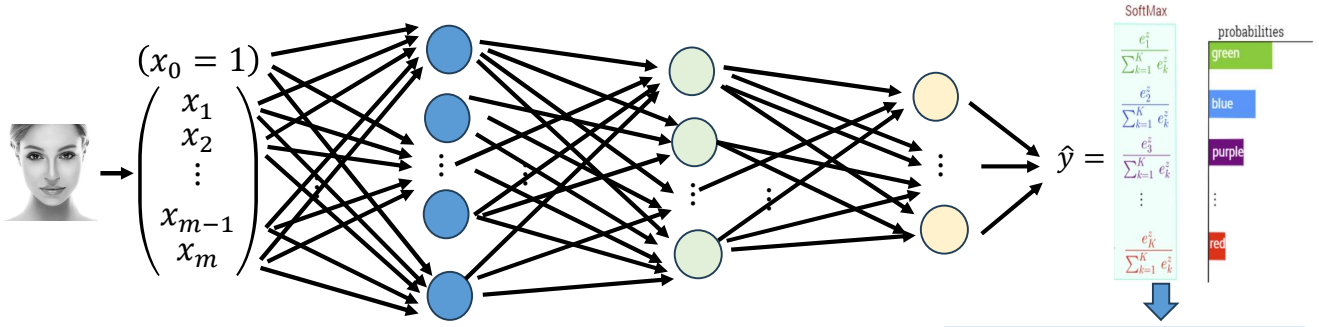


SWINBURNE UNIVERSITY OF TECHNOLOGY

38

Welcome to Artificial Neural Networks (ANNs)



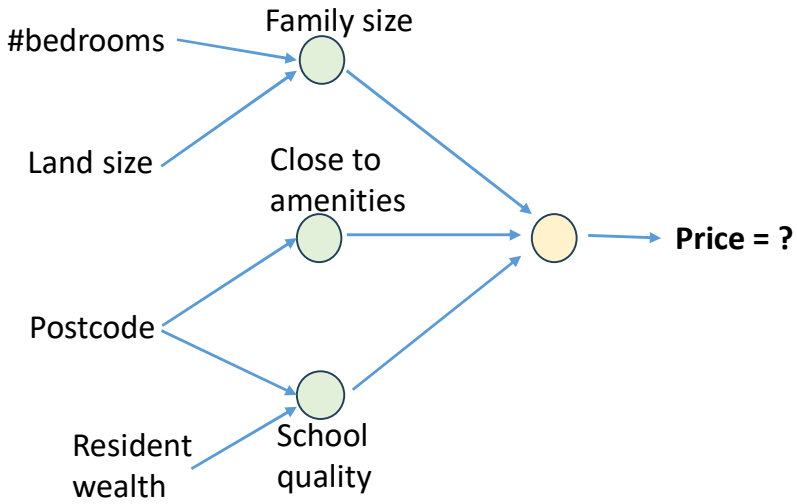


It's **Taylor**! **72%** certain!

39

(Automatic) Feature Engineering

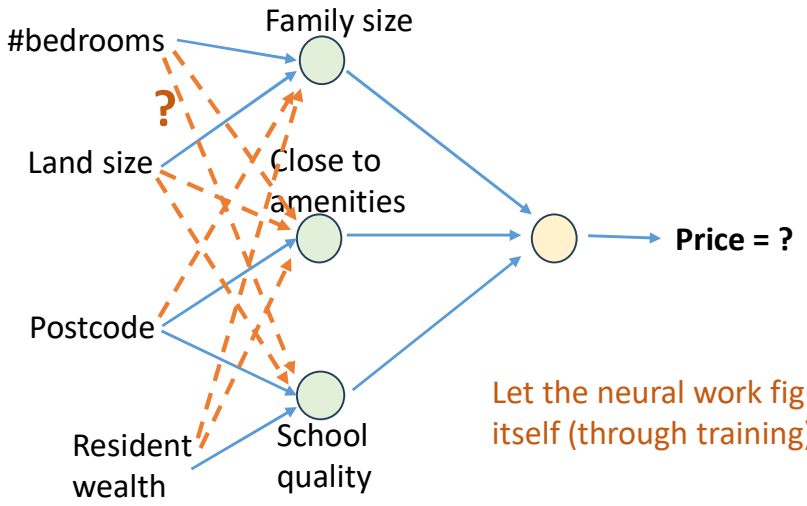
Task: Predict the house price.



40

(Automatic) Feature Engineering

Task: Predict the house price.

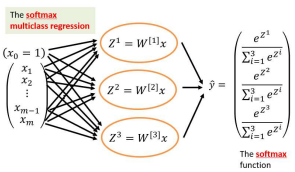


Let the neural work figure this question out by itself (through training)

41

Important observation

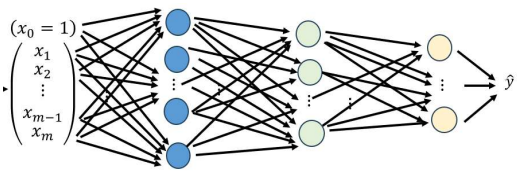
- There can be many ML problems/tasks but (most of) them can be dealt with by neural networks of different architectures (after a training process)

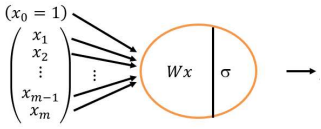


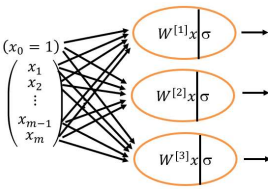
Artificial Neural Network (ANN)/model = **architecture** + **parameters**

ML practitioners

Trained with data



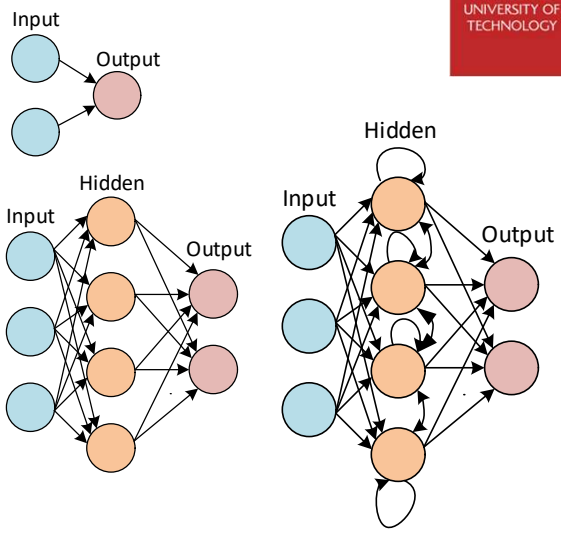




42

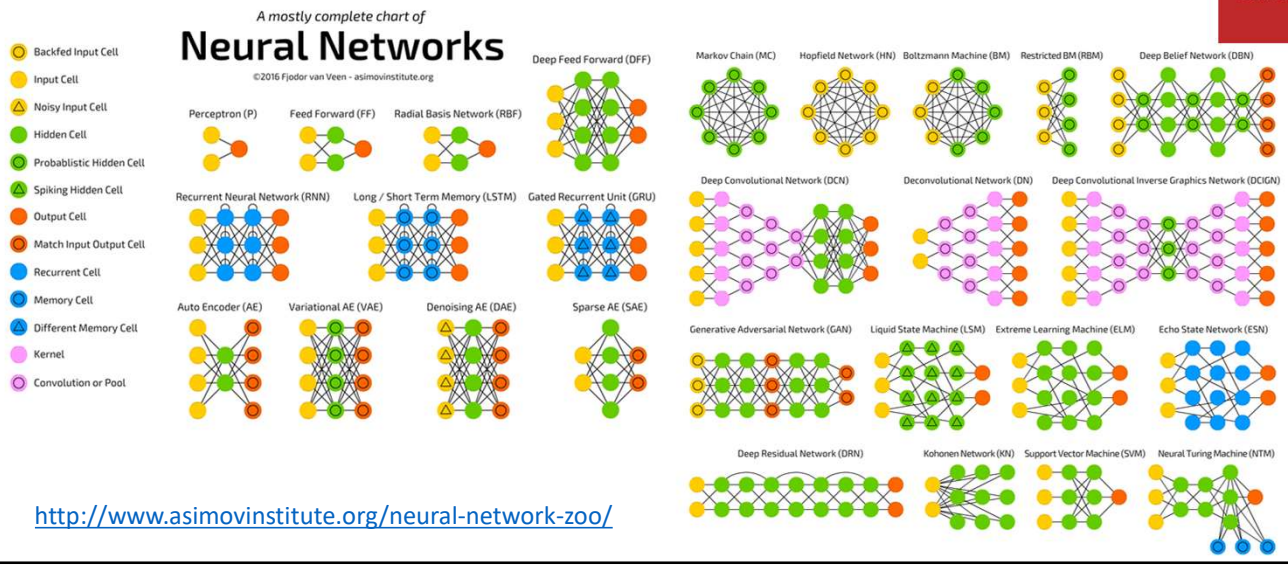
Complex Neural Network - Architectures

- There are 2 main categories of neural network structures:
  - Acyclic/Feed-Forward Networks – no loops, no internal state
    - Single-layer perceptron
    - Multi-layer perceptron
  - Cyclic/Recurrent Networks – feeds its outputs back into its own inputs, directed cycles, supports short-term memory



43

Types of Neural Networks

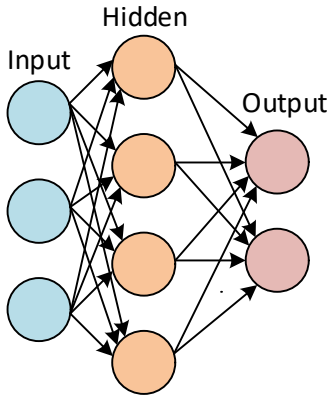


http://www.asimovinstitute.org/neural-network-zoo/

44

Multiple-Layer Neural Networks

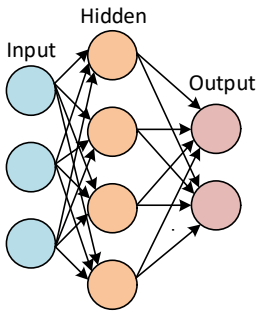
- A complex neural network involves a set of neurons (e.g. perceptrons) interconnected in layers to solve more complex problems
  - Input layer neurons connect to hidden layer neurons
  - Hidden layer neurons connect to output layer neurons
  - Output layer can have multiple outputs
- Multilayer neural networks can classify a range of functions including non-linearly separable ones



45

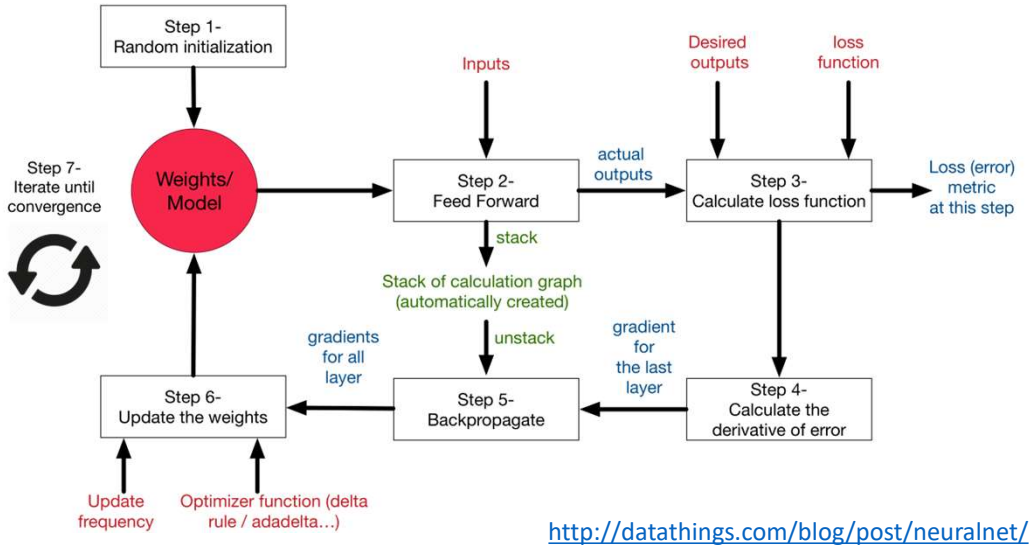
Learning in Multiple-Layer Neural Networks

- Multilayer NN learn the same way as perceptrons
- **Forward pass** – compute the values from input to output
- **Backward pass** – propagate errors backwards to adjust the connection weights
- Error occurs when expected output != actual output
- **Cost/Loss function** – a measure of "how good or bad" a NN did w.r.t its given training sample and the expected output



46

Learning in Multiple-Layer Neural Networks



http://datathings.com/blog/post/neuralnet/

47

Backpropagation

- An approach for training a ANN
  - used to optimize weights in a multi-layer NN so that it can learn to correctly map arbitrary inputs to outputs
  - Objective of training the network is to *minimize* value of the *cost function* by adjusting the weights

$$W := W - \alpha \frac{\partial J}{\partial W}$$

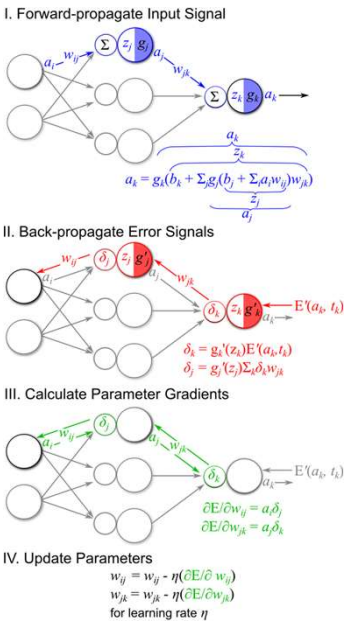
- α – learning rate, W - weight, J – cost function
- $\frac{\partial J}{\partial W}$ - Partial derivative of J w.r.t W

NOTE: partial derivative of a function of several variables is its derivate with respect to one variable keeping every other variable constant

48

Back Propagation Algorithm

- **Feedforward** – input signals are forward propagated though the network towards the outputs
  - The output is a composite function of the weights, inputs and activation function(s)
- **Verification** - Actual output value is compared with expected output.
  - ERROR = Desired output – Actual output
- **Backpropagation** – Network errors are back propagated backwards towards to the inputs



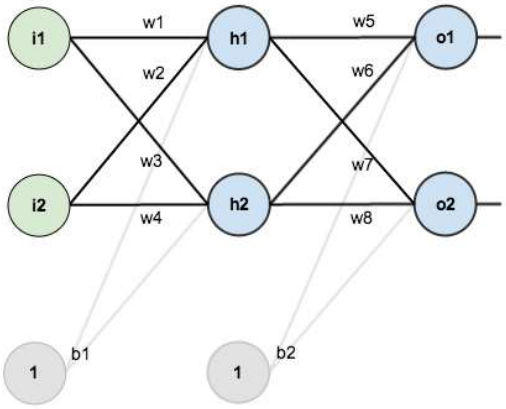
https://theclevermachine.wordpress.com/2014/09/11/a-gentle-introduction-to-artificial-neural-networks/

49

Back Propagation – Step by Step Example

Neural Network Structure

- Neural network with two inputs, two hidden neurons, and two output neurons
- Hidden and output neurons have a bias

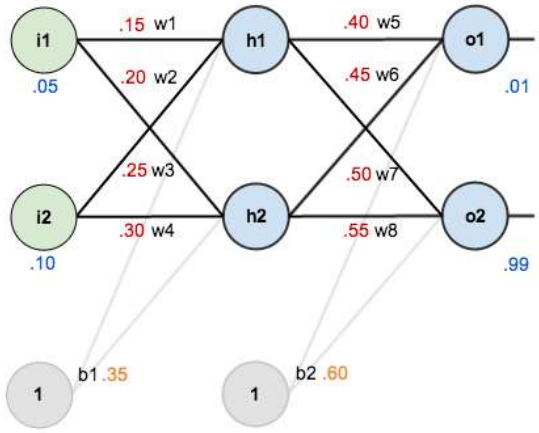


https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

50

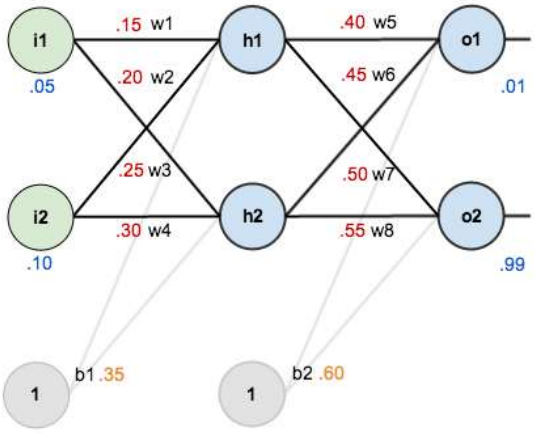
Back Propagation – Step by Step Example

Initial Weights, Biases and Training Inputs/Outputs



51

Back Propagation – Step by Step Example



Goal of Back Propagation

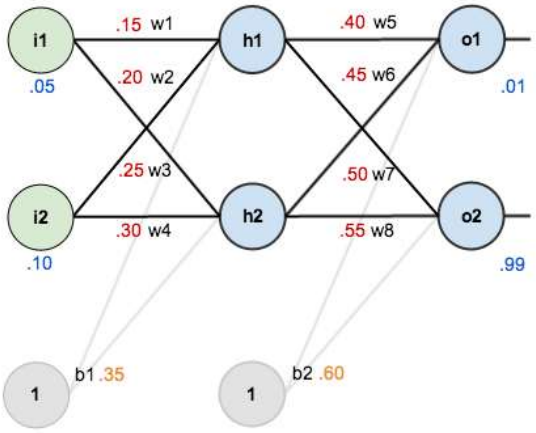
- Optimize weights so that neural network can learn to correctly map arbitrary inputs to outputs
- Given the inputs 0.05 and 0.10, the neural network should output 0.01 and 0.99

52

Back Propagation – Step by Step Example

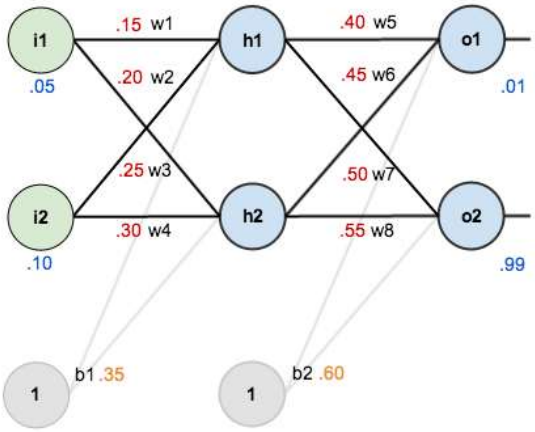
Forward Pass

- Feed the inputs forward through the network
- Compute net input to each hidden neuron, apply activation function to it, repeat process with output layer neurons



53

Back Propagation – Step by Step Example



Forward Pass – 1:

Calculate total net input net_{h1} for *h1*

$net_{h1}=w_1*i_1 + w_2*i_2 + b_1$

$net_{h1}= 0.15*0.05+0.2*0.1+0.35$

$net_{h1}= 0.3775$

Apply activation function to calculate output out_{h1} of *h1*

$out_{h1} = \frac{1}{1+e^{-neth_1}}$ (logistic function)

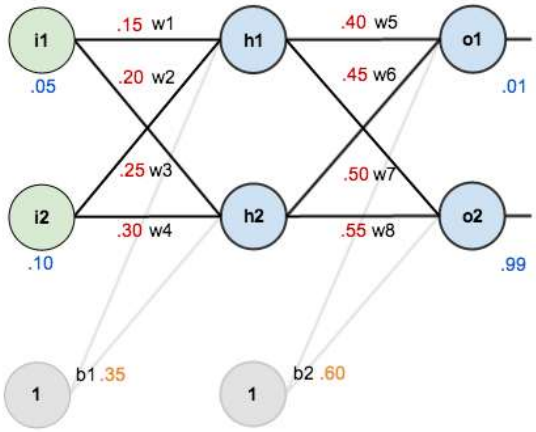
$= \frac{1}{1+e^{-0.3775}} = 0.59329992$

Do the same for h_2

$out_{h2} = 0.596884378$

54

Back Propagation – Step by Step Example



Forward Pass - 2

Calculate net input net_{01} for *o1*

$net_{o1}=w_5*out_{h1} + w_6*out_{h2} + b_2$

$= 0.40* 0.59329992 +0.45* 0.596884378 + 0.60$

$= 1.105905967$

Apply activation function to calculate output out_{01} of *o1*

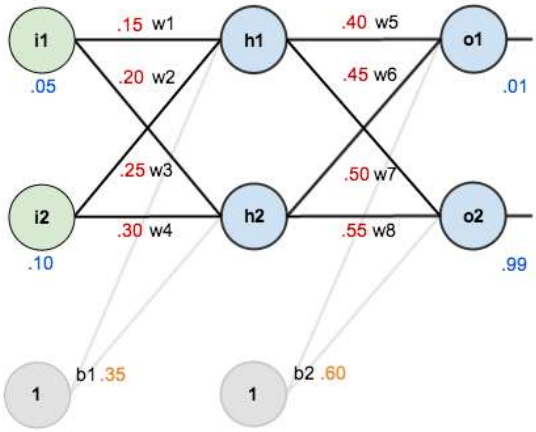
$out_{o1} = 0.75136507$

Do the same for o_2

$out_{o2} = 0.772928465$

55

Back Propagation – Step by Step Example



Calculate Error

Calculate the total error of each output neuron using the *Squared Error Function*

$$E = \sum \frac{1}{2}(target - output)^2$$

Error E_{01} for output neuron *o1*

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2$$

$$E_{o1} = \frac{1}{2}(0.01 - 0.7514)^2$$

$$E_{o1} = 0.274811083$$

Error E_{o2} for output neuron *o2*

$$E_{o2} = 0.023560026$$

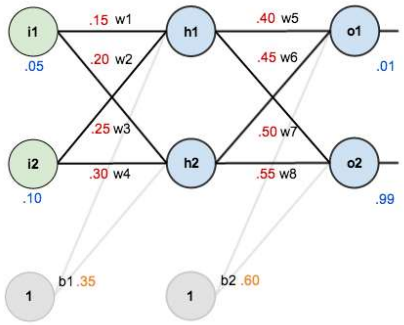
Total Error is the sum of the errors

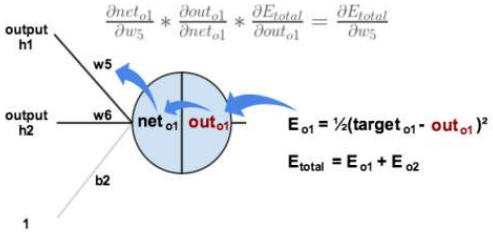
$$E_{total} = 0.274811083 + 0.023560026$$

$$= 0.298371109$$

56

Back Propagation – Step by Step Example





Backward Pass

We want to determine how much a change in w_5 contributes to the total error E_{total}

$$\frac{\partial Etotal}{\partial w_5} = \frac{\partial Etotal}{\partial outo_1} * \frac{\partial outo_1}{\partial neto_1} * \frac{\partial neto_1}{\partial w_5}$$

How much does the error change w.r.t out_{01}?

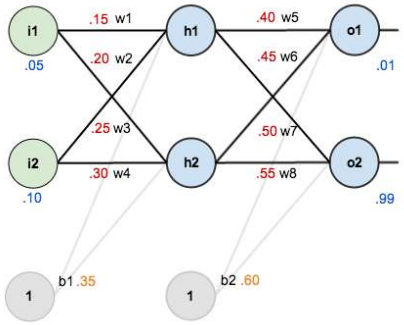
$$E_{total} = \frac{1}{2}(target_{o1} \_ out_{o1})^2 + \frac{1}{2}(target_{o2} \_ out_{o2})^2$$

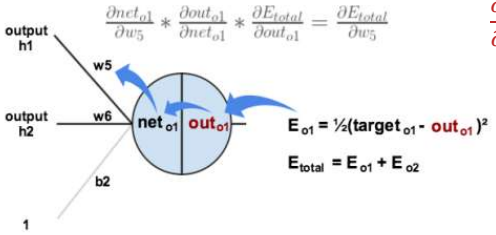
$$\frac{\partial Etotal}{\partial outo_1} = 2 * \frac{1}{2}(target_{o1} \_ out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial Etotal}{\partial outo_1} = -(target_{o1} \_ out_{o1})$$

$$\frac{\partial Etotal}{\partial outo_1} = -(0.01 - 0.75136507) = 0.74136507$$

Back Propagation – Step by Step Example





Backward Pass

We want to determine how much a change in w_5 contributes to the total error E_{total}

$$\frac{\partial Etotal}{\partial w_5} = \frac{\partial Etotal}{\partial outo_1} * \frac{\partial outo_1}{\partial neto_1} * \frac{\partial neto_1}{\partial w_5}$$

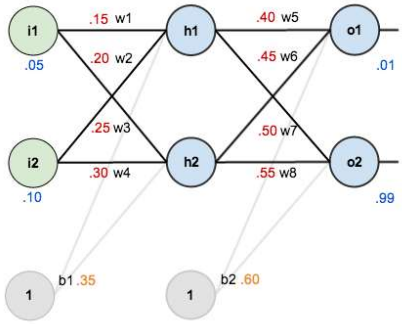
How much does the output out_{01} change w.r.t net_{01}?

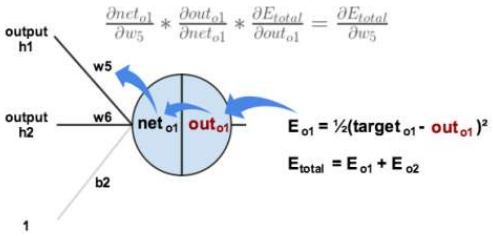
$$out_{o1} = \frac{1}{1 + e^{-neto_1}}$$

$$\frac{\partial ou_{o1}}{\partial net_{o1}} = out_{o1}(1 \_ outo_1) = 0.775136507\ (1 - 0.75136507)$$

$$\frac{\partial out_{o1}}{\partial net_{o1}} = 0.186815602$$

Back Propagation – Step by Step Example





Backward Pass

We want to determine how much a change in w_5 contributes to the total error E_{total}

$$\frac{\partial Etotal}{\partial w_5} = \frac{\partial Etotal}{\partial outo_1} * \frac{\partial outo_1}{\partial neto_1} * \frac{\partial neto_1}{\partial w_5}$$

How much does the net input net_{o1} change w.r.t w_5?

$$net_{o1} = w_5 * outh_1 + w_6 * outh_2 + b_2 * 1$$

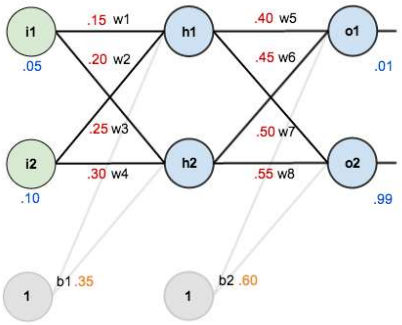
$$\frac{\partial neto_1}{\partial w_5} = out_{h1} + 0 + 0 = 0.593269992$$

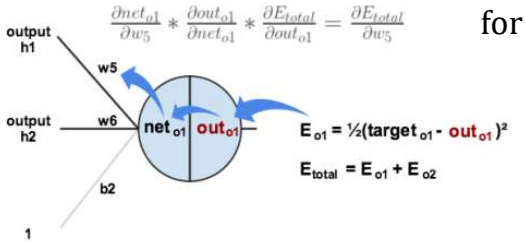
$$\frac{\partial Etotal}{\partial w_5} = 0.74136507 * 0.186815602 * 0.59327 = 0.082167041$$

SWINBURNE UNIVERSITY OF TECHNOLOGY

59

Back Propagation – Step by Step Example





Update weight

Calculating new weight (using an optional learning rate α of 0.5)

$$w_5^+ = w_5 - \alpha * \frac{\partial Etotal}{\partial w_5}$$

$$w_5^+ = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

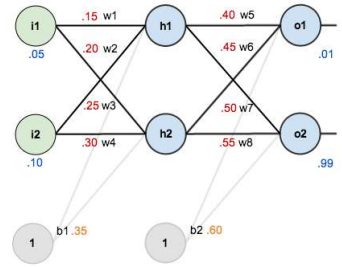
Similarly, we can calculate new weights for w_6, w_7 and w_8

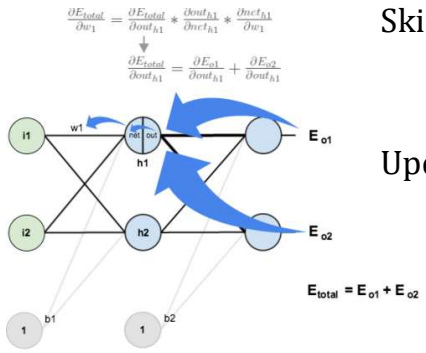
$$w_6^+ = 0.408666186$$
$$w_7^+ = 0.511301270$$
$$w_8^+ = 0.561370121$$

SWINBURNE UNIVERSITY OF TECHNOLOGY

60

Back Propagation – Step by Step Example





Backward Pass - 2

We calculate the new value for the weight w_1, w_2, w_3, w_4

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$

Skipping a few steps….

$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 * 0.241300709 * 0.05 = 0.000438568$$

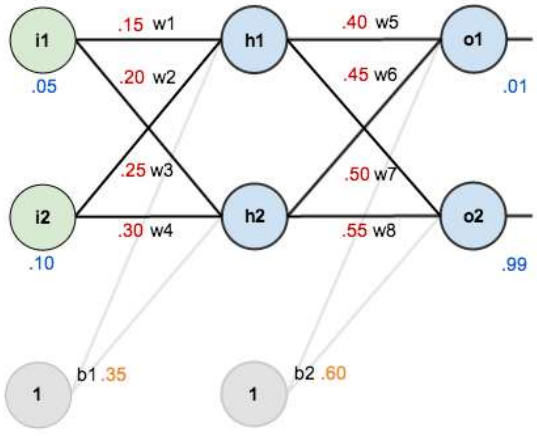
Updating the weights…. $w_1^+ = w_1 - \alpha * \frac{\partial Etotal}{\partial w_1}$

$$w_1^+ = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

$$w_2^+ = 0.19956143, \quad w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

Back Propagation – Step by Step Example



- After updating all the weights, the total error of the network is reduced to 0.298371109
- The original error was 0.291027924
- If this process is repeated 10000 times, the error is reduced to 0.0000351085
- At this point, the inputs of 0.05 and 0.1 produce outputs of 0.015912196 (vs 0.01 target) and 0.984065734 (vs 0.99 target)

Developing a Neural Network

- Needs a fair degree of mathematical competence
- Not simple (also very time consuming)
- Considerations include:
  - Identification of features in the problem domain worth mapping into inputs/outputs (Problem Representation)
  - Determining the number of hidden layers and nodes
  - Setting up initial parameters – weights, thresholds, increments
  - What learning method to use?
- No rules exist, based on experience (and solid understanding of problem domain)

63

Weakness of Neural Networks

- Neural networks are *opaque* – it is very hard to check that the weights after training are sensible
- A neural network can never explain how it arrived at a particular conclusion
- Loss of human control, lack of trust since AI cannot explain all decisions and actions, or take responsibility – **Black Box AI**
- Some work is being done on combating the opaqueness problem
- Partnership on AI (https://www.partnershiponai.org) is a step towards safe and trustworthy AI technologies – supported by tech giants including Google, IBM and Microsoft

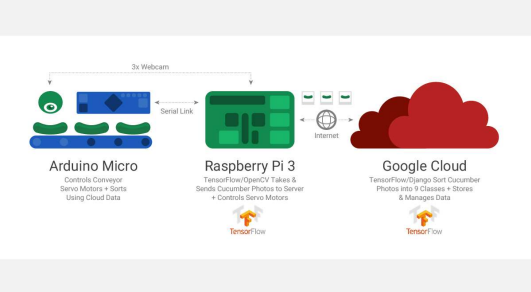
64

Machine Learning as a Service

- Machine learning is no longer for experts only
- Software developers can use cutting-edge, commercially usable machine learning frameworks
  - Scikit-learn
  - Google Tensorflow
  - PyTorch
  - Microsoft Cognitive Toolkit
  - Theano
  - Caffe
  - Deeplearning4j
- Major cloud providers including Amazon, Google and Microsoft offer machine learning as a service

65

Cucumber Farming & TensorFlow



- Cucumber classification into 9 different classes using Tensorflow based on size, thickness, color, texture, small scratches, whether they are crooked, whether they have prickles,

https://cloud.google.com/blog/big-data/2016/08/how-a-japanese-cucumber-farmer-is-using-deep-learning-and-tensorflow

66